

Introduction to Cryptography and Security

Exercise 13: Message Authentication Codes

December 10, 2025

Suppose a MAC system (S, V) is used to protect files in a file system by appending a MAC tag to each file. The MAC signing algorithm S is applied to the file contents and nothing else. What tampering attacks are not prevented by this system?

- 1 Swapping two files in the file system.
- 2 Replacing the tag and contents of one file with the tag and contents of a file from another computer protected by the same MAC system, but a different key.
- 3 Replacing the contents of a file with the concatenation of two files on the file system.
- 4 Erasing the last byte of the file contents.

Let (S, V) be a secure MAC defined over (K, M, T) where $M = \{0, 1\}^n$ and $T = \{0, 1\}^{128}$.

Consider the following MAC construction:

$$\begin{aligned} S'(k, m) &= S(k, m[0, \dots, n-2] \parallel 0) \\ V'(k, m, t) &= V(k, m[0, \dots, n-2] \parallel 0, t). \end{aligned}$$

That is, the signing algorithm S' takes a message m and effectively ignores its last bit by setting it to 0 before applying the original secure MAC signing function S . The verification algorithm V' similarly ignores the last bit of m .

Is this a secure MAC?

Let (S, V) be a secure MAC defined over (K, M, T) where $M = \{0, 1\}^n$ and $T = \{0, 1\}^{128}$.

Consider the following MAC construction:

$$S'(k, m) = S(k, m)[0, \dots, 126]$$
$$V'(k, m, t) = [V(k, m, t \parallel 0) \text{ or } V(k, m, t \parallel 1)]$$

That is, the signing algorithm S' truncates the tag generated by the original secure MAC S , effectively losing one bit of information. The verification algorithm V' accepts if either of the two possible original tags (with the last bit 0 or 1) is valid.

Is this a secure MAC?

Let (S, V) be a secure MAC defined over (K, M, T) where $M = \{0, 1\}^n$ and $T = \{0, 1\}^{128}$.

Consider the following MAC construction:

$$\begin{aligned} S'(k, m) &= S(k, m \parallel m) \\ V'(k, m, t) &= V(k, m \parallel m, t). \end{aligned}$$

Is this a secure MAC?

Let (S, V) be a secure MAC defined over (K, M, T) where $M = \{0, 1\}^n$ and $T = \{0, 1\}^{128}$.

Consider the following MAC construction:

$$S'(k, m) = S(k, m) \quad \text{and} \\ V'(k, m, t) = \begin{cases} V(k, m, t) & \text{if } m \neq 0^n \\ \text{"1"} & \text{otherwise} \end{cases}$$

Is this a secure MAC?

Let (S, V) be a secure MAC defined over (K, M, T) where $M = \{0, 1\}^n$ and $T = \{0, 1\}^{128}$.

Consider the following MAC construction:

$$S'(k, m) = S(k, m)$$

$$V'(k, m, t) = [V(k, m, t) \text{ or } V(k, m \oplus 1^n, t)]$$

Is this a secure MAC?

Let (S, V) be a secure MAC defined over (K, M, T) where $M = \{0, 1\}^n$ and $T = \{0, 1\}^{128}$.

Consider the following MAC construction:

$$S'((k_1, k_2), m) = (S(k_1, m), S(k_2, m))$$
$$V'((k_1, k_2), m, (t_1, t_2)) = [V(k_1, m, t_1) \text{ and } V(k_2, m, t_2)]$$

Is this a secure MAC?

Recall that the ECBC-MAC uses a fixed IV (in the lecture we simply set the IV to 0). If a random IV is chosen for each message and included in the tag, i.e., $S(k, m) := (r, \text{ECBC}_r(k, m))$ where $\text{ECBC}_r(k, m)$ uses r as the IV, and verification $V(k, m, (r, t))$ outputs “1” if $t = \text{ECBC}_r(k, m)$ and “0” otherwise.

The resulting MAC system is insecure. An attacker can query for the tag of a 1-block message m to get (r, t) . He can then generate the following existential forgery:

- 1 The tag $(r, t \oplus r)$ is a valid tag for the 1-block message 0^n .
- 2 The tag $(m \oplus t, t)$ is a valid tag for the 1-block message 0^n .
- 3 The tag $(r \oplus 1^n, t)$ is a valid tag for the 1-block message $m \oplus 1^n$.
- 4 The tag $(m \oplus t, r)$ is a valid tag for the 1-block message 0^n .

Consider the encrypted CBC MAC built from AES. Suppose we compute the tag for a long message m comprising of n AES blocks. Let m' be the n -block message obtained from m by flipping the last bit of m (i.e. if the last bit of m is b then the last bit of m' is $b \oplus 1$).

How many calls to AES would it take to compute the tag for m' from the tag for m and the MAC key? (in this question please ignore message padding and simply assume that the message length is always a multiple of the AES block size)

- ① 4
- ② 5
- ③ 2
- ④ n

The message X to be authenticated consists of z independent blocks such that $X = x_1 || x_2 || \dots || x_z$, where every x_i consists of $|x_i| = 8$ bits. The input blocks are consecutively put into a compression function $h()$ that works as follows:

$$c_i = h(c_{i-1}, x_i) = c_{i-1} \oplus x_i$$

with $c_0 = 0$. Both the c_i and the x_i are 8-bit values. The MAC is computed from the last output c_z of the compression function:

$$S(k, X) \equiv c_z + k \pmod{2^8}$$

where k is a 64-bit shared key.

Describe how exactly the (effective part of the) key k can be calculated with only one known message X .

- Perform this attack for the following values and determine the key k :

$X = \text{HELLO ALICE!}$

$c_0 = 1111\ 1111_2$

$S(k, X) = 1001\ 1101_2$

- What is the effective key length of k ?

The CBC-MAC construction builds a MAC function from a block cipher by taking the last encrypted block of the CBC-mode encryption.

Can we similarly invent an ECB-MAC or an OFB-MAC and obtain secure constructions?